

An Automatic Parking System in Unity based on Reinforcement Learning

Yue Xue 20406680

1. Introduction

With the development of autonomous car technology, vehicles could make more decisions according to the feedback from the surrounding environments without the control of human drivers. The implementation of autonomous vehicles will reduce the traffic accident fatality rate and traffic congestion (Van Brummelen et al., 2018). Pan et al. (2017) suggest that it is not appropriate to use reinforcement learning in autonomous driving technology in a real vehicle environment. RL algorithms need to learn from zero through rewards and feedbacks and that will significantly increase the risks of traffic accidents. On the other hand, training autonomous vehicle model in virtual environments using RL algorithm is a widely accepted option (Pan et al., 2017; Sutton et al., 2018). Automatic Parking System (APS) is one area of autonomous vehicle (AV) technology. An APS needs the vehicle to collect surrounding information through sensors such as light detection and ranging (LiDAR), cameras and radar sensors, etc. The agent which is the vehicle should make decisions about the next movement based on the input data until park into the parking slot safely. There has been an increasing interest in using reinforcement learning (RL) to solve APS problems (Takehara and Gonsalves, 2021; Zhang, 2020). Unity provides a virtual environment for simulation of the agents.

This research utilizes the ML-Agents framework of Unity to conduct simulation training experiments. The trainings enable the AI to learn to park a car in the correct location in a parking lot in a 3D physics simulation environment. The AI consists of a deep neural network, each network with two hidden layers, and each layer containing 128 neurons. The network is trained by the nearest neighbor policy optimization (PPO) algorithm of RL. The inputs to the neural network are the readings from the pivotal sensors, including the current speed and position of the car, and its relative position to the target. This research designs an APS using RL algorithm in Unity. This study will train the model with RL algorithm in the environment at first, and then compare the performance of the model in a sparse parking lot and a dense parking lot by changing the number of unmoved vehicles. This system will be trained in a dense parking lot with a few free slots and a sparse parking lot with many free slots. The metrics of the study are the accumulative rewards and the entropy.

2. Related Work

Many researches have proposed new APS prototypes based on RL algorithms.

Thunyapoo et al. (2020) choose the Unity virtual environment for creating their APS model using RL. This research categorizes the RL algorithms into value-based methods and actor-critic methods. They argue that value-based methods such as deep Q-learning (DQN) are suitable for the discrete actions and not good for continuous movements. This research uses one actor-critic RL algorithm called proximal policy optimisation (PPO). This study suggests that PPO is suitable for creating a robust APS model and the model could learn from the environment in a moderate speed. Three different reward settings are designed in the experiments. The results show that dense reward setting could achieve better performance. A dense reward means the agent is provided with reward feedback at every step, while a sparse reward defines that the agent is given rewards when a certain percentage of distance decreases between the agent and the parking slot. The proposed model achieves a high success parking rates of over 90% in the basic area while receives low success parking rates of less than 10%. The experiment environment is shown as Figure 1.

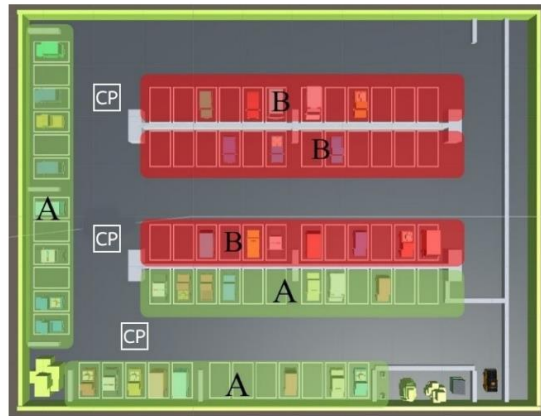


Figure 1. Parking lot with basic slots Zone A and challenging slots Zone B (Thunyapoo et al., 2020)

Takehara and Gonsalves (2021) also use PPO to develop an APS model in Unity. Different from the previous research, this study categorizes RL algorithms as value-based methods and policy-based methods. This research argues that the policy-based algorithm such as PPO will not occupy too much memory like the value-based algorithm DQN. The purpose of this research is to use the onboard cameras in the vehicle to complete the APS model, and this design could reduce the production costs of the system. Cumulative rewards and entropies are the evaluation metrics of the learning. Three kinds of experiment environments are set in this research. One parking lot does not have any obstacle, the second parking lot have some static obstacles and the final parking lot have some moving obstacles. This research receives good results in all of the three experiments and proves that the vehicle could reach parking slot accurately and smoothly with only onboard cameras.

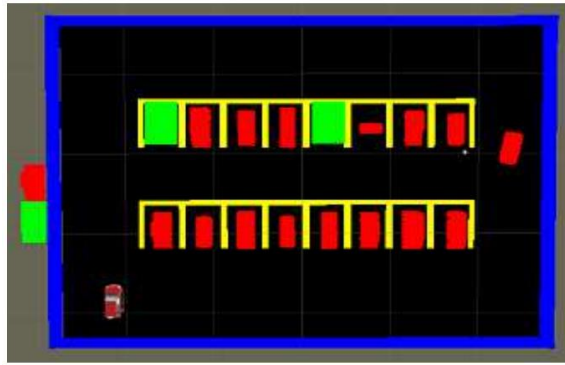


Figure 2. Parking lot with moving obstacles (Takehara and Gonsalves, 2021)

In the process of creating new APS models, the experiment in a virtual environment is only the beginning. After the simulation step, tests in hardware are needed (Du et al., 2020), and then experiments of real vehicles in the real parking lot are required (Zhang et al., 2019). It is better to have benchmark tests with other APS products used in the industry to evaluate the performance of the proposed prototype.

3. Methodology

3.1 Unity Machine Learning Agents

Machine Learning Agents (ML-Agents) is an open-source machine learning plug-in used in Unity. It is a framework that can be great for getting started with deep learning and reinforcement learning because learners could exactly see what's going on in the machine learning process, not just numbers and letters on the terminal. The environmental scenarios used by machine learning have several features such as high visual complexity (e.g., Dota2 professional players), high physical complexity (e.g., quadrupedal creature walking) and high cognitive complexity (e.g., AlphaGo). The above scenarios are difficult to solve using traditional algorithms, but it is easier to solve these problems using machine learning. Using the ML-Agents plug-in, some intelligent operations of game objects can be realized. Examples include controlling NPC behavior (with various settings such as multiple agents and confrontation), automating testing of game builds, and evaluating pre-release builds for different game design decisions, etc. RL is one of the most used learning algorithms in ML-Agents (Valecillos, 2020).

3.2 Reinforcement Learning

Reinforcement learning (RL) is one kind of machine learning methods. RL has several characteristics such as learning by rewards, trial and error and time dependent. RL requires the training object to constantly interact with the environment, and to sum up the best behavioral decision at each step through trial and error. There is no guidance in the whole process, only cold feedback. RL trains subjects to adjust their behavioral decisions based on environmental feedbacks. Delayed feedback is one

characteristic of RL. During the RL training process, the trial-and-error behavior of the training object obtains feedbacks from the environment. Sometimes the agent will wait until the entire training is over and then get feedbacks. A series of environmental state changes and environmental feedback in RL are strongly linked to time. The entire reinforcement RL process is time-dependent. The state and feedback are also constantly changing. The current behavior affects subsequent data received. The current state and the action taken will affect the state received in the next step. There is a certain correlation between data and data in RL. This is different from the other machine learning (ML) algorithms like supervised ML or unsupervised RL (Sutton and Barto, 2018).

As shown in the figure 3, the agent returns the incentive (state or reward) according to the agent's behavior (action) through the external environment. After the agent receives the incentive, it changes its behavior and feeds it back to the external environment, thereby form a closed-loop learning process. As discussed in literatures, there are two kinds of RL algorithms, which are value-based algorithm and policy-based algorithm (i.e., actor critic algorithm). PPO is a good choice for the parking project because it has smooth speed and will not take up too much memory of the computer.

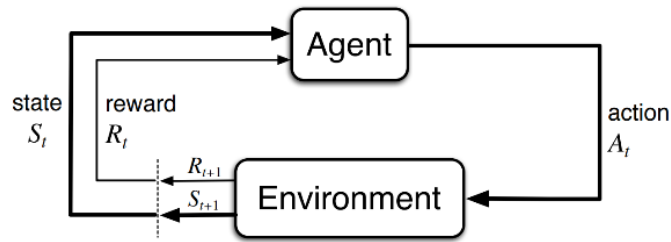


Figure 3. The agent–environment interaction in a Markov decision process (Sutton and Barto, 2018)

3.3 Evaluation Metrics

There are two kinds of metrics in this research, which are the cumulative rewards and the entropy. The model is good if it reaches a high reward value and a low entropy value. Rewards mean the feedbacks which the agent receives from the environment after actions in the current state. The cumulative rewards show the summation of the agent's performance in all steps and episodes. The entropy is given by the following equation (Takehara and Gonsalves, 2021).

$$-\sum_i P(A_i) \log P(A_i)$$

4 Experiments

4.1 Experimental Setup

A. Environment

This research utilizes the environment created by Valecillos (2020) using Unity 2019.3.5f1 version. The training is conducted using ML-Agents 0.29.0.dev0 version and PyTorch 1.11.0 version. The program is mainly implemented in C#.

As shown in Figure 4 and 8. The parking lot has fifteen parking slots. Four kinds of obstacles such as barriers, trees, cars and the ground exist in the environment. The road is on the bottom of the environment. When the training begins, nine parking lots with the same kind of agents would be trained together so as to speed up the training process as shown in Figure 5 and 9. The parameters of this experiments are shown in Table 1.

Table 1. Parameters of the Experiments

Name	Value
Speed	0.5
Torque	20
Min Torque	0.1
Min Idle	1
Max Step	2000
Rays Per Direction	8
Ray Radius	0.2
Ray Vertical Offset Start	0.2
Decision Period	5
Goal Min Rotation	0.3
Goal Reward	2
Reward Penalty	-0.01f
Iteration	430000

B. Agent

The agent of this research is a car model. The agent has position, rotation, velocity and torque information. The agent has five different states include turning left, turning right, moving forward, moving backward and staying idle. During the training process, the velocity of the agent needs to be controlled since it might not learn in a too slow or too fast velocity. When the car reaches the target parking lot successfully, that slot will turn to green. However, if the car does not park into the empty slot successfully, the area will become red and the agent will come back to default (Valecillos, 2020).

The agent is embedded with a pivotal sensor. The sensor is just like the LiDAR in real life, which enables the agent to see the surrounding environment such as the barriers, the slot and the other vehicles. The sensor is located on top of the agent, and the parameters of the sensor is shown in Table 1.

C. Reward Setting

The reward will be given after every action so that the agent could move towards the goal car slot. The step numbers of one episode could be set. In this research one episode contains 1000 steps, and the training is supposed to complete in 5000 episodes. The reward setting value is shown in Table 1.

4.2 Dense Parking Lot

The first training is conducted in a dense parking lot with up to three free slots randomly appears in the lot. The rewards performance is shown in Figure 6 and the entropy is in Figure 7. Figure 4 and 5 are taken from Unity, and Figure 6 and 7 are taken from Tensor board. After five million cycles of training in the dense parking lot, the accumulative rewards reach 2.4 and the entropy is around 0.1.



Figure 4. A dense lot in the training

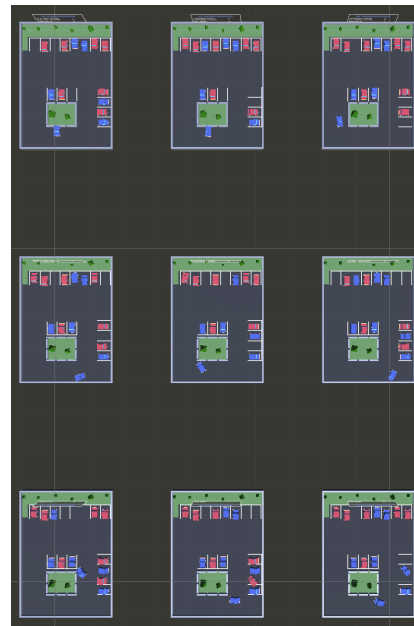


Figure 5. Nine dense lots in the training

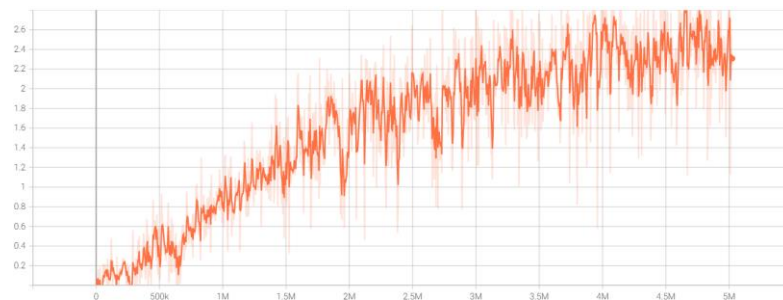


Figure 6. Cumulative Rewards in a dense lot



Figure 7. Entropy in a dense lot

4.3 Sparse Parking Lot

The second training is conducted in a sparse parking lot with up to three free slots randomly appears in the lot. The rewards performance is shown in Figure 10 and the entropy is in Figure 11. Figure 8 and 9 are taken from Unity, and Figure 10 and 11 are taken from Tensor board. After five million cycles of training in the sparse parking lot, the accumulative rewards reach 3.6 and the entropy is below 0.1.



Figure 8. A sparse lot during the in training Figure 9. Nine sparse lots in the training

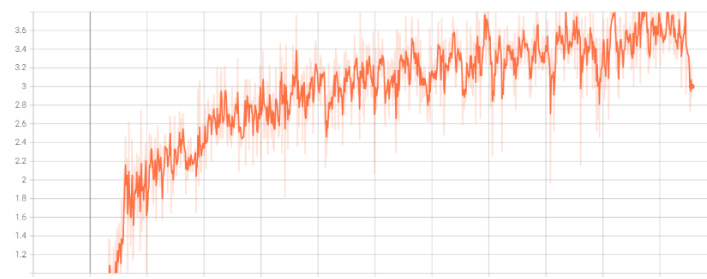


Figure 10. Cumulative Rewards in a sparse lot



Figure 11. Entropy in a sparse lot

4.4 Discussion

The trainings in both a dense lot and a sparse lot have been conducted over five million cycles. The agent is awarded if it parks the car well or moving close to the target slot and punished if it drives away or crashes. The experimental environments in the two kinds of lots are almost same. The only difference is that in the dense parking lot as shown in Figure 4 and 5, there are one to three free lots available randomly in the whole lot. However, in the sparse parking lot such as Figure 8 and 9, there are up to ten free lots.

In comparison with Figure 6 and 10, the accumulative reward in the sparse lot is 50% higher than that in the dense lot. In comparison with Figure 7 and 11, the entropy in the sparse lot is lower than that in the dense lot. Therefore, the agent performs better in a sparse lot than in a dense lot. It might be because the agent has more choices in the sparse lot and could reach the free parking slot more quickly and easily.

5 Conclusion

This research introduces a very basic automatic parking model with a simple environment. The models are trained in a dense parking lot and a sparse lot. There are two tasks of this model have not been finished yet.

The first unfinished work is the dashboard of the parking lot. A dashboard is designed in each parking lot to show the real-time information of cumulative rewards, completed episodes and step counts. However, currently only a dashboard exists while the information could not be update simultaneously.

The other unfinished work is the goal type of the reward settings. The agent is rewarded based on its distance with the target parking slot, i.e., the rewards will be given if its distance decreases while the penalties will be given if the distance increases. However, if there are several empty parking slots, this agent currently could not calculate its distances with every empty lots. At this moment, there is only one target parking lot in each slot. Therefore, if you pay attention to the nine parking lots in the screen, you could notice that each agent would continue to move into one specific parking slot during each process, and the nine agents in different slots are moving to their own target slots. In the future work, the agent should be able to calculate its distance and rewards with every empty parking slot and be able to park into every empty slots randomly. The goals of rewarding settings should be put in every empty slots rather than only one parking slot. The dashboard could be put on every empty slot and show the cumulative rewards between the agent and this specific slot.

This research provides a software prototype, and test it in the virtual environment of Unity. As for the future work, the code could be implemented in some hardware and have a real vehicle benchmark tests with the APS models in the market.

Reference

- Du, Z., Miao, Q. and Zong, C. (2020) Trajectory Planning for Automated Parking Systems Using Deep Reinforcement Learning. **International Journal of Automotive Technology** 21(4): pp.881-887.
- Pan, X., You, Y., Wang, Z. and Lu, C. (2017) Virtual to real reinforcement learning for autonomous driving. **arXiv preprint** arXiv:1704.03952.
- Sutton, R.S. and Barto, A.G. (2018) **Reinforcement learning: an introduction**. Second edition. Boston: The MIT Press.
- Takehara, R. and Gonsalves, T. (2021) Autonomous Car Parking System using Deep Reinforcement Learning. **In 2021 2nd International Conference on Innovative and Creative Information Technology (ICITech)** pp.85-89.
- Thunyapoo, B., Ratchadakorntham, C., Siricharoen, P. and Susutti, W. (2020) Self-Parking Car Simulation using Reinforcement Learning Approach for Moderate Complexity Parking Scenario. **In 2020 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)** pp.576-579.
- Valecillos, D. (2020) **Github: UnityMLEssentials**[online]. Available at: <https://github.com/dilmerv/UnityMLEssentials> [Accessed 10 May 2022].
- Valecillos, D. (2020) **YouTube: How To Park A Car With Unity Machine Learning ML-Agents** [online]. Available at: https://youtu.be/k_JNyLoB8vg [Accessed 10 May 2022].
- Van Brummelen, J., O'Brien, M., Gruyer, D. and Najjaran, H. (2018) Autonomous vehicle perception: The technology of today and tomorrow. **Transportation research part C: emerging technologies** 89: pp.384-406.
- Zhang, J., Chen, H., Song, S. and Hu, F. (2020) Reinforcement learning-based motion planning for automatic parking system. **IEEE Access** 8: pp.154485-154501.
- Zhang, P., Xiong, L., Yu, Z., Fang, P., Yan, S., Yao, J. and Zhou, Y. (2019) Reinforcement learning-based end-to-end parking for automatic parking system. **Sensors** 19(18): pp.3996.